



Swift Tuition's Swift EO3 ~~Study Guide~~ ~~CheatSheet~~ Neither Both?



Swift Heads Up (I Love Abusing The Word Swift Like This 🤪)

Usually I'd make both a study guide and a cheat sheet, but as I only had time to make one, I left them alone in a room for not even a minute and their child (this document) was the result.

It feels too short to be a study guide, but too long to be a cheat sheet, so let's consider it both?

Also, I know y'all find me hilarious and look forward to finding something funny on the second page of the study guides, but I didn't have time to think up something funny. 😊

BUT wouldn't it be funny if on the second page of the cheat sheet I made a meta reference TO the fact that I didn't have the time to type something funny on the second page of the cheat sheet? And wouldn't the longer I spend typing about how I didn't have time to spend coming up with something funny be eating into the time I COULD have spent coming up with something funny? Also I'm currently in this grey zone where I don't know if the longer I ramble about how I don't have time to come up with something funny makes this more funny or less funny.

Wait, why did I think this was a good idea? Why can't I stop? Why am I still typing this? I'm not thinking any longer, I'm just typing the first words to come to mind. Must stop. Can't Stop. Must keep typing - CAN'T STOP TYPING...sidfhsfhadhuisdauigfduisdafusdauisfdauiosdfvusvdafo.

Great, finally snapped out of it.

In a very smooth transition... should you find yourself academically slipping and may need some "Swift" help in time for the supps (2025 humans)....<https://www.swifttuition.com/course-detail/cos-110-eo3-programme/34572/> 🤖 - your contribution towards the Feed the Me Foundation will be immeasurable and greatly appreciated. 🙏 (if you're reading this after 2025, everything you need can be found here: www.swifttuition.com/tuks/cos110)

"I feel like an undertaker waiting for people to die so I can make money 😊 - I OBVIOUSLY wish you all the best BUUUUT - should the things that push begin to shove...you know where to find me 😊 Some of the vids and notes shall be free preview to allow me to prove myself first 😊

I've rambled enough (I could have used that time to think of something funny 😊) - anywho, 'twas a tassel 'twixt AI and I conjuring up these. Pick what you feel you need reminder of.

Enjoy 🤪. (can you tell Roald Dahl was my favourite author in primary?? 😊)



O. FOUNDATION QUICK REFERENCE

Not the main focus of EO3, but you need to know these basics. Quick syntax reminders only.

POINTERS

A pointer stores a memory address. Think of it as "pointing to" a variable.

Basic Syntax:

```
int x = 5;
int* ptr;    // Declare pointer to int
ptr = &x;    // ptr now points to x (& = address-of)
cout << ptr;  // Prints address (e.g., 0x7fff5fbff5ac)
cout << *ptr; // Prints 5 (dereference - get value at address)
*ptr = 10;    // Changes x to 10 (through pointer)
```

Key operators: `&` = address-of operator (get address of variable) | `*` = dereference operator (get value at address)

NULL Pointers:

```
int* ptr = NULL;    // Points to nothing (C++98)
int* ptr = nullptr; // C++11 way

// Always check before using!
if (ptr != NULL) {
    cout << *ptr;    // Safe
}
```

Why NULL check? Dereferencing NULL pointer = segmentation fault (crash)

Pointer Arithmetic:

```
int arr[5] = {10, 20, 30, 40, 50};
int* ptr = arr;    // Points to first element
ptr++;             // Points to next element (arr[1])
ptr--;             // Points to previous element
ptr += 2;          // Jump 2 elements forward
cout << *ptr;       // Value at current position
cout << *(ptr + 3); // Value 3 positions ahead
```

Pointers to Structures/Classes:

```
struct Node {
    int data;
    Node* next;
};

Node* ptr = new Node;
ptr->data = 5;    // Access member (arrow operator)
(*ptr).data = 5; // Same thing (verbose)
```

Arrow operator `->` is shorthand for `(*ptr).`



CLASSES

Basic Class Structure:

```
class Rectangle {
private:
    int width, height;    // Can only be accessed inside class
public:
    Rectangle() { width = 0; height = 0; } // Constructor
    Rectangle(int w, int h) { width = w; height = h; }
    ~Rectangle() { /* Cleanup code here */ } // Destructor

    int area() { return width * height; }
    void setWidth(int w) { width = w; }
    int getWidth() const { return width; } // const = doesn't modify
};

// Using the class
Rectangle r1;           // Calls default constructor
Rectangle r2(5, 10);    // Calls parameterised constructor
r2.setWidth(7);
cout << r2.area();      // 70
```

Access Specifiers:

```
class Example {
private:
    int x;    // Only accessible inside class
protected:
    int y;    // Accessible in class and derived classes
public:
    int z;    // Accessible everywhere
};
```

The this Pointer:

```
class Box {
private:
    int value;
public:
    void setValue(int value) {
        this->value = value; // this->value is member
    }
    Box* getThis() { return this; } // Returns pointer to this object
};
```

this is a pointer to the current object.

Copy Constructor:

```
class MyClass {
private:
    int* data;
    int size;
public:
    MyClass(const MyClass& other) { // Copy constructor
        size = other.size;
        data = new int[size];
        for (int i = 0; i < size; i++) {
            data[i] = other.data[i]; // Deep copy
        }
    }
    ~MyClass() { delete[] data; }
};
```

Deep copy vs shallow copy: Shallow: Copy pointer value (both point to same data) | Deep: Copy data itself (each has own copy)

Assignment Operator:

```
class MyClass {
public:
    MyClass& operator=(const MyClass& other) {
        if (this != &other) { // Check for self-assignment
            delete[] data;    // Free old data
            size = other.size;
            data = new int[size];
            for (int i = 0; i < size; i++) {
                data[i] = other.data[i];
            }
        }
        return *this; // Return reference to this object
    }
};
```

TEMPLATES

Templates let you write code that works with any type.

Function Templates:

```
template <class T>
T maximum(T a, T b) {
    return (a > b) ? a : b;
}

// Or using typename (same thing)
template <typename T>
T minimum(T a, T b) {
    return (a < b) ? a : b;
}

// Usage
int x = maximum(5, 10);    // T is int
double y = maximum(3.5, 2.1); // T is double
string s = maximum("apple", "banana"); // T is string
```

class T and typename T are the same thing in templates.

Class Templates:

```
template <class T>
class Box {
private:
    T value;
public:
    Box(T val) { value = val; }
    T getValue() { return value; }
    void setValue(T val) { value = val; }
};

// Usage - must specify type
Box<int> intBox(5);
Box<double> doubleBox(3.14);
Box<string> stringBox("hello");
cout << intBox.getValue(); // 5
```

Multiple Template Parameters:

```
template <class T, class U>
class Pair {
private:
    T first;
    U second;
public:
    Pair(T f, U s) : first(f), second(s) {}
    T getFirst() { return first; }
    U getSecond() { return second; }
};

// Usage
Pair<int, string> p(1, "apple");
Pair<double, char> p2(3.14, 'A');
```

Template Member Functions (Outside Class):

```
template <class T>
class Stack {
private:
    T* array;
    int top;
public:
    void push(T item); // Declaration
    T pop();
};

// Definition OUTSIDE class - need template header again
template <class T>
void Stack<T>::push(T item) {
    /* implementation */
}

template <class T>
T Stack<T>::pop() {
    /* implementation */
}
```

Important: Template code usually goes in .h files, not .cpp!



DYNAMIC MEMORY ALLOCATION

new and delete:

```
// Single object
int* ptr = new int;    // Allocate one int
*ptr = 5;
delete ptr;           // Free memory
ptr = NULL;           // Good practice

// With initialisation
int* ptr = new int(10); // Allocate and initialise to 10

// Array
int* arr = new int[10]; // Allocate array of 10 ints
arr[0] = 5;
delete[] arr;          // Note the []! Must use for arrays
arr = NULL;

// Object
Node* node = new Node;
node->data = 5;
delete node;

// Array of objects
Node* nodes = new Node[5];
delete[] nodes;
```

Swift Note: `new` → must use `delete` | `new[]` → must use `delete[]` (with brackets!) | Don't mix them up!

Memory Leaks:

```
// BAD - memory leak!
void leak() {
    int* ptr = new int(5);
    // Function ends, ptr is lost, memory never freed!
}

// GOOD - free memory
void noLeak() {
    int* ptr = new int(5);
    delete ptr; // Memory freed
}

// BAD - lose pointer before deleting
Node* head = new Node;
head = new Node; // Lost old node! Memory leak!

// GOOD - delete before reassigning
Node* head = new Node;
delete head;
head = new Node;
```

ARRAYS

Static Arrays:

```
int arr[5];           // Uninitialised
int arr[5] = {1, 2, 3, 4, 5}; // Initialised
int arr[] = {1, 2, 3}; // Size inferred (3)
arr[0] = 10;
cout << arr[2];

// Size known at compile time
int size = 5;
int arr[size]; // Error in C++98! Size must be constant

const int SIZE = 5;
int arr[SIZE]; // OK - SIZE is constant
```

Passing Arrays to Functions:

```
// Arrays passed as pointers - size lost!
void print(int arr[], int size) { // arr[] same as int*
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
}

// Must pass size separately
int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    print(arr, 5); // Array "decays" to pointer
}
```

Key point: When you pass array to function, it becomes a pointer. Size information is lost!

2D Arrays:

```
// Static 2D array
int matrix[3][4]; // 3 rows, 4 columns

// Initialise
int matrix[3][4] = {{1, 2, 3, 4},
{5, 6, 7, 8},
{9, 10, 11, 12}};

// Access
matrix[0][0] = 1; // First element
matrix[2][3] = 12; // Last element

// Dynamic 2D array
int rows = 3, cols = 4;
int** matrix = new int*[rows]; // Array of pointers
for (int i = 0; i < rows; i++) {
    matrix[i] = new int[cols]; // Each pointer gets an array
}
matrix[0][0] = 1; // Access same way

// Must delete in reverse order
for (int i = 0; i < rows; i++) {
    delete[] matrix[i];
}
delete[] matrix;
```



STRUCTS vs CLASSES

```
// Struct - members public by default
struct Point {
    int x;
    int y;
}; // Public

// Class - members private by default
class Point {
    int x;
    int y;
}; // Private
```

Main difference: Default access level. Otherwise identical!

PASS BY VALUE vs REFERENCE vs POINTER

```
void byValue(int x) { x = 10; }    // Only changes local copy
void byReference(int& x) { x = 10; } // Changes original!
void byPointer(int* x) { *x = 10; } // Changes original!
void constRef(const int& x) { cout << x; } // Can't modify, but no copy

int main() {
    int a = 5;
    byValue(a);    // a still 5
    byReference(a); // a now 10
    byPointer(&a);  // a now 10
    constRef(a);   // a unchanged
}
```

When to use what: Value: Small types (int, char), don't need to modify | Reference: Large types (objects), need to modify | Const reference: Large types, don't need to modify (efficient!) | Pointer: Dynamic memory, optional parameter (can be NULL)

COMMON OPERATORS

```
// Arithmetic: a + b, a - b, a * b, a / b, a % b
// Increment/Decrement: ++a (pre-increment), a++ (post-increment), --a, a--
// Comparison: a == b, a != b, a < b, a > b, a <= b, a >= b
// Logical: a && b (AND), a || b (OR), !a (NOT)
// Bitwise: a & b, a | b, a ^ b, ~a, a << n, a >> n
// Assignment: a = b, a += b, a -= b, a *= b, a /= b, a %= b
```

COMMON HEADER FILES

```
#include <iostream> // cout, cin
#include <string>    // string class
#include <vector>    // vector container
#include <list>      // list container
#include <stack>     // stack adapter
#include <queue>     // queue, priority_queue
#include <map>       // map, multimap
#include <set>       // set, multiset
#include <algorithm> // sort, find, etc.
#include <cmath>     // sqrt, pow, etc.
#include <cstdlib>   // NULL, rand, etc.

using namespace std; // So you don't need std::cout
```

NAMESPACE

```
// Without using namespace std
std::cout << "Hello" << std::endl;
std::vector<int> v;
std::string s = "hello";

// With using namespace std
using namespace std;
cout << "Hello" << endl;
vector<int> v;
string s = "hello";

// Selective using
using std::cout;
using std::endl;
cout << "Hello" << endl;
```




1. LINKED LISTS

A linked list is a data structure made up of nodes. Each node has data and a pointer to the next node. Unlike arrays, linked lists don't store elements in one block of memory - each node can be anywhere in memory, connected by pointers.

Advantages over arrays:

- Can grow/shrink during runtime (dynamic size)
- Easy to insert/delete in middle (just change pointers)
- No wasted memory from pre-allocated space

Disadvantages:

- No random access (can't jump to index 5 directly)
- Extra memory for pointers
- Must traverse from head to find elements

SINGLY LINKED LISTS

Each node points to the next node. The last node points to NULL.

Node Structure:

```
struct Node {
    int data;    // Can be any type
    Node* next; // Pointer to next node
};
```

You can also have a node class inside a list class:

```
class LinkedList {
private:
    class Node {
    public:
        int data;
        Node* next;
        Node() { data = 0; next = NULL; }
        Node(int val) { data = val; next = NULL; }
    };
    Node* head; // Pointer to first node
public:
    LinkedList() { head = NULL; } // Start with empty list
};
```

The Head Pointer:

The head pointer is your entry point to the list. If head == NULL, the list is empty.



Important RULE:

Never modify the head pointer when traversing! If you lose head, you lose the entire list.

```
// WRONG - loses the list!
while (head != NULL) {
    cout << head->data;
    head = head->next; // DANGER! head is moving
}

// CORRECT - use a temporary pointer
Node* current = head;
while (current != NULL) {
    cout << current->data;
    current = current->next; // Safe - head unchanged
}
```

NULL Checks - The Most Important Pattern

Before you access ANY pointer, check if it's NULL. This prevents segmentation faults.

```
// Check if list is empty
if (head == NULL) {
    cout << "List is empty";
    return;
}

// Check if at last node
if (current->next == NULL) {
    cout << "This is the last node";
}

// Before dereferencing ANY pointer
if (ptr != NULL) {
    cout << ptr->data; // Safe
}

// If you don't check
cout << ptr->data; // CRASH if ptr is NULL!
```

When to check for NULL: Before accessing `ptr->data` | Before accessing `ptr->next` | At the start of any function that works with nodes | In loop conditions

Traversing a Linked List:

```
void printList() {
    if (head == NULL) {
        cout << "Empty list" << endl;
        return;
    }

    Node* current = head;
    while (current != NULL) {
        cout << current->data << " ";
        current = current->next;
    }
    cout << endl;
}
```

Why this works: Start at head (first node) | Process current node | Move to next node (`current = current->next`) | Stop when current becomes NULL (end of list)

Counting nodes:

```
int countNodes() {
    int count = 0;
    Node* current = head;
    while (current != NULL) {
        count++;
        current = current->next;
    }
    return count;
}
```

Finding a node with specific value:

```
Node* find(int value) {
    Node* current = head;
    while (current != NULL) {
        if (current->data == value) {
            return current; // Found it!
        }
        current = current->next;
    }
    return NULL; // Not found
}
```

Getting the last node:

```
Node* getTail() {
    if (head == NULL) return NULL; // Empty list

    Node* current = head;
    while (current->next != NULL) {
        current = current->next;
    } // Stop at last node
    return current;
}
```



Inserting Nodes

Insert at Head (Front of List)

This is the EASIEST insert because you don't need to traverse.

```
void insertAtHead(int value) {
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = head; // Link to rest FIRST
    head = newNode;      // THEN update head
}
```

❑ **Why this order matters:** If you do `head = newNode` first, you lose the old list! Always link the new node BEFORE updating head.

Insert at Tail (End of List):

```
void insertAtTail(int value) {
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = NULL; // It will be the last node

    if (head == NULL) {
        head = newNode;
        return;
    } // Special case: empty list

    // Find last node
    Node* current = head;
    while (current->next != NULL) {
        current = current->next;
    }
    current->next = newNode; // Link last node to new node
}
```

Why check for empty list? If `head == NULL`, there's no node to traverse to. The new node becomes the head.

Alternative with tail pointer:

```
class LinkedList {
private:
    Node* head;
    Node* tail; // Keep track of last node
public:
    void insertAtTail(int value) {
        Node* newNode = new Node(value);
        if (head == NULL) {
            head = tail = newNode; // First node
        } else {
            tail->next = newNode;
            tail = newNode;
        }
    }
};
```

Insert at Specific Position (by index):

```
void insertAt(int index, int value) {
    if (index == 0) {
        insertAtHead(value);
        return;
    } // Special case: insert at head

    // Find node BEFORE insertion point
    Node* current = head;
    for (int i = 0; i < index - 1 && current != NULL; i++) {
        current = current->next;
    }

    if (current == NULL) {
        cout << "Index out of bounds" << endl;
        return;
    } // Check if position exists

    // Insert new node
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = current->next; // Link to rest of list first
    current->next = newNode;      // Then link previous node to new node
}
```

Swift Note: You need the node BEFORE where you want to insert (at position `index - 1`).

Insert After a Specific Node:

```
void insertAfter(Node* prevNode, int value) {
    if (prevNode == NULL) {
        cout << "Previous node cannot be NULL" << endl;
        return;
    }

    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = prevNode->next; // Link new node to the rest of the list
    prevNode->next = newNode;      // Link previous node to new node
}
```

Insert in Sorted Position:

```
void insertSorted(int value) {
    Node* newNode = new Node;
    newNode->data = value;

    // Case 1: Empty list or insert at head
    if (head == NULL || value < head->data) {
        newNode->next = head;
        head = newNode;
        return;
    }

    // Case 2: Find correct position
    Node* current = head;
    while (current->next != NULL && current->next->data < value) {
        current = current->next;
    }

    // Insert after current
    newNode->next = current->next;
    current->next = newNode;
}
```




Deleting Nodes

Delete from Head:

```
void deleteHead() {
    if (head == NULL) {
        cout << "List is empty" << endl;
        return;
    }

    Node* temp = head;    // Save current head
    head = head->next;    // Move head to next node
    delete temp;        // Free memory
}
```

Why use temp? If you delete head directly, you can't access head->next. Save it first, move head, THEN delete.

Delete from Tail:

```
void deleteTail() {
    if (head == NULL) {
        cout << "List is empty" << endl;
        return;
    }

    // Special case: only one node
    if (head->next == NULL) {
        delete head;
        head = NULL;
        return;
    }

    // Find second-to-last node
    Node* current = head;
    while (current->next->next != NULL) {
        current = current->next;
    }

    delete current->next; // Delete last node
    current->next = NULL;
}
```

Why find second-to-last? You need to set the new last node's next to NULL. Can't do that from the last node itself.

Delete Node with Specific Value:

```
void deleteValue(int value) {
    if (head == NULL) return;

    // Special case: delete head
    if (head->data == value) {
        Node* temp = head;
        head = head->next;
        delete temp;
        return;
    }

    // Find node BEFORE the one to delete
    Node* current = head;
    while (current->next != NULL && current->next->data != value) {
        current = current->next;
    }

    if (current->next == NULL) {
        cout << "Value not found" << endl;
        return;
    } // Check if found

    // Delete the node
    Node* temp = current->next;
    current->next = current->next->next; // Bridge around
    delete temp;
}
```

Delete at Specific Position:

```
void deleteAt(int index) {
    if (head == NULL) return;

    // Delete head
    if (index == 0) {
        Node* temp = head;
        head = head->next;
        delete temp;
        return;
    }

    // Find node before deletion point
    Node* current = head;
    for (int i = 0; i < index - 1 && current != NULL; i++) {
        current = current->next;
    }

    if (current == NULL || current->next == NULL) {
        cout << "Index out of bounds" << endl;
        return;
    }

    // Delete node
    Node* temp = current->next;
    current->next = current->next->next;
    delete temp;
}
```

Edge Cases for Singly Linked Lists:

Empty List (head == NULL)

- Insert: New node becomes head
- Delete: Nothing to delete (just return)
- Search: Return NULL or -1

Single Node List

- Delete head: Set head = NULL after deleting
- Delete tail: Same as delete head
- Insert after: Check if next == NULL

Deleting Last Node

- Need to find second-to-last node
- Set its next to NULL

Value Not Found

- Search operations should return NULL or indicator
- Delete operations should notify user

Inserting at Invalid Position

- Check if position exists before inserting
- Handle gracefully (return or throw exception)



DOUBLY LINKED LISTS

Each node has TWO pointers: one to next node, one to previous node. This allows traversal in both directions.

Node Structure:

```
struct Node {
    int data;
    Node* next; // Points to next node
    Node* prev; // Points to previous node
};
```

Doubly Linked List Class:

```
class DoublyLinkedList {
private:
    class Node {
    public:
        int data;
        Node* next;
        Node* prev;
        Node(int val = 0) {
            data = val;
            next = NULL;
            prev = NULL;
        }
    };
    Node* head;
    Node* tail; // Often track both ends
public:
    DoublyLinkedList() {
        head = NULL;
        tail = NULL;
    }
};
```

Advantages:

- Can traverse backwards
- Delete a node without finding previous
- Insert before a node easier

Disadvantages:

- More memory (extra pointer per node)
- More complex (must maintain two pointers)
- More operations (every change affects two pointers)

Traversing:

```
// Forward traversal (same as singly)
void printForward() {
    Node* current = head;
    while (current != NULL) {
        cout << current->data << " ";
        current = current->next;
    }
    cout << endl;
}

// Backward traversal
void printBackward() {
    Node* current = tail;
    while (current != NULL) {
        cout << current->data << " ";
        current = current->prev;
    }
    cout << endl;
}
```

Insert at Head:

```
void insertAtHead(int value) {
    Node* newNode = new Node(value);

    if (head == NULL) {
        head = tail = newNode;
        return;
    } // Empty list

    newNode->next = head;
    head->prev = newNode; // Link head back to new node
    head = newNode; // Update head
}
```

Note: Must update BOTH next and prev pointers.

Insert at Tail:

```
void insertAtTail(int value) {
    Node* newNode = new Node(value);

    if (tail == NULL) {
        head = tail = newNode;
        return;
    } // Empty list

    newNode->prev = tail;
    tail->next = newNode;
    tail = newNode; // Update tail
}
```

Insert After a Specific Node:

```
void insertAfter(Node* prevNode, int value) {
    if (prevNode == NULL) {
        cout << "Previous node cannot be NULL" << endl;
        return;
    }

    Node* newNode = new Node(value);
    newNode->next = prevNode->next;
    newNode->prev = prevNode;

    if (prevNode->next != NULL) {
        prevNode->next->prev = newNode; // Update next node's prev (if exists)
    } else {
        tail = newNode; // Inserting at tail
    }

    prevNode->next = newNode; // Link previous node to new node
}
```

☒ **Four pointer changes!** `newNode->next = prevNode->next` | `newNode->prev = prevNode` | `prevNode->next->prev = newNode` (if not at tail) | `prevNode->next = newNode`

Insert Before a Specific Node

EASIER with doubly linked lists because the node knows its previous!

```
void insertBefore(Node* nextNode, int value) {
    if (nextNode == NULL) {
        cout << "Next node cannot be NULL" << endl;
        return;
    }

    Node* newNode = new Node(value);
    newNode->next = nextNode;
    newNode->prev = nextNode->prev;

    if (nextNode->prev != NULL) {
        nextNode->prev->next = newNode; // Update previous node's next (if exists)
    } else {
        head = newNode; // Inserting at head
    }

    nextNode->prev = newNode; // Link next node to new node
}
```

Delete a Specific Node (Given the Node)

This is where doubly linked lists shine! No need to find the previous node.

```
void deleteNode(Node* delNode) {
    if (delNode == NULL) return;

    // Update previous node's next
    if (delNode->prev != NULL) {
        delNode->prev->next = delNode->next;
    } else {
        head = delNode->next; // Deleting head
    }

    // Update next node's prev
    if (delNode->next != NULL) {
        delNode->next->prev = delNode->prev;
    } else {
        tail = delNode->prev; // Deleting tail
    }

    delete delNode;
}
```

Four cases to handle: Deleting head (prev == NULL) | Deleting tail (next == NULL) | Deleting middle node | Deleting only node (both NULL)

Delete from Head:

```
void deleteHead() {
    if (head == NULL) return;

    Node* temp = head;
    head = head->next;

    if (head != NULL) {
        head->prev = NULL; // New head has no previous
    } else {
        tail = NULL; // List is now empty
    }

    delete temp;
}
```

Delete from Tail:

```
void deleteTail() {
    if (tail == NULL) return;

    Node* temp = tail;
    tail = tail->prev;

    if (tail != NULL) {
        tail->next = NULL; // New tail has no next
    } else {
        head = NULL; // List is now empty
    }

    delete temp;
}
```



CIRCULAR LINKED LISTS

In a circular linked list, the last node points back to the first node instead of NULL. There's no "end" to the list - it loops forever.

Why Use Circular Lists?

- Round-robin scheduling
- Game turns
- Buffer management
- Music playlists

Circular List Structure:

```
class CircularLinkedList {
private:
    Node* tail; // Usually track tail, not head
public:
    CircularLinkedList() { tail = NULL; }
    // tail->next gives you the head!
};
```

Why track tail instead of head? tail->next gives you head for free | Inserting at tail AND head both become O(1)

Key Difference: No NULL Checks:

```
// WRONG for circular lists
while (current->next != NULL) { // Will never be NULL!
    current = current->next;
}

// CORRECT for circular lists
Node* start = tail->next; // Remember where we started
Node* current = start;
do {
    // Process current
    current = current->next;
} while (current != start); // Stop when back at start
```

Traversing:

```
void printList() {
    if (tail == NULL) {
        cout << "Empty list" << endl;
        return;
    }

    Node* current = tail->next; // Start at head
    do {
        cout << current->data << " ";
        current = current->next;
    } while (current != tail->next); // Back to head
    cout << endl;
}
```

Insert at Head (using tail pointer):

```
void insertAtHead(int value) {
    Node* newNode = new Node(value);

    if (tail == NULL) {
        tail = newNode;
        newNode->next = newNode; // Empty list - node points to itself
        return;
    }

    newNode->next = tail->next; // Link to current head
    tail->next = newNode;      // Tail now points to new head
    // Note: tail doesn't move!
}
```

Insert at Tail (using tail pointer):

```
void insertAtTail(int value) {
    Node* newNode = new Node(value);

    if (tail == NULL) {
        tail = newNode;
        newNode->next = newNode;
        return;
    } // Empty list

    newNode->next = tail->next; // Link to head
    tail->next = newNode;      // Old tail links to new node
    tail = newNode;           // Update tail
}
```

Only difference from insertAtHead: Last line moves tail!

Delete from Head:

```
void deleteHead() {
    if (tail == NULL) return; // Empty list

    if (tail == tail->next) {
        delete tail;
        tail = NULL;
        return;
    } // Only one node

    // Multiple nodes
    Node* temp = tail->next; // Save head
    tail->next = temp->next;  // Tail points to new head
    delete temp;
}
```

Delete from Tail:

```
void deleteTail() {
    if (tail == NULL) return;

    if (tail == tail->next) {
        delete tail;
        tail = NULL;
        return;
    } // Only one node

    // Find node before tail
    Node* current = tail;
    while (current->next != tail) {
        current = current->next;
    }

    current->next = tail->next; // Link to head
    delete tail;
    tail = current;           // Update tail
}
```

Delete Specific Value:

```
void deleteValue(int value) {
    if (tail == NULL) return;

    // Check if only one node
    if (tail == tail->next && tail->data == value) {
        delete tail;
        tail = NULL;
        return;
    }

    // Find node to delete and the one before it
    Node* current = tail;
    Node* toDelete = tail->next; // Start at head

    do {
        if (toDelete->data == value) {
            current->next = toDelete->next;
            if (toDelete == tail) {
                tail = current; // If deleting tail, update tail pointer
            }
            delete toDelete;
            return;
        }
        current = toDelete;
        toDelete = toDelete->next;
    } while (toDelete != tail->next);

    cout << "Value not found" << endl;
}
```

Edge Cases for Circular Lists:

Empty List

tail == NULL (or head == NULL)
First insertion: node points to itself

Single Node

tail->next == tail (points to itself)
Deletion: set tail = NULL

Finding the "End"

No NULL to mark end
Must check if you've looped back to start

Traversal Infinite Loops

If you forget to check for return to start
Will loop forever!



2. STACKS

A stack is a Last-In-First-Out (LIFO) data structure. Think of a stack of plates - you can only add or remove from the top.

Stack Operations:

- `push(item)` add to top
- `pop()` remove and return top
- `top()/peek()` view top without removing
- `isEmpty()`
- `isFull()` (array only)
- `size()`

When to Use Stacks:

- Undo/Redo
- Function calls (the call stack)
- Expression evaluation
- Backtracking
- Browser history
- Parsing (balanced parentheses)

ARRAY-BASED STACK

Fixed size stack using an array.

```
template <class T>
class Stack {
private:
    T* array;
    int stackTop; // Index of top element
    int maxSize;
public:
    Stack(int size = 100) {
        maxSize = size;
        array = new T[maxSize];
        stackTop = -1; // -1 means empty
    }

    ~Stack() {
        delete[] array;
    }

    bool isEmpty() const {
        return (stackTop < 0);
    }

    bool isFull() const {
        return (stackTop == maxSize - 1);
    }

    int size() const {
        return stackTop + 1;
    }

    void push(const T& item) {
        if (isFull()) {
            throw "Stack Overflow";
        }
        array[++stackTop] = item; // Pre-increment!
    }

    T pop() {
        if (isEmpty()) {
            throw "Stack Underflow";
        }
        return array[stackTop--]; // Post-decrement!
    }

    T top() const {
        if (isEmpty()) {
            throw "Stack is empty";
        }
        return array[stackTop];
    }

    void clear() {
        stackTop = -1;
    }
};
```

Key Details:

stackTop index: -1 means empty (no elements) | 0 means 1 element at index 0 | `maxSize - 1` means full

Pre-increment vs Post-decrement:

```
// Push: pre-increment (++stackTop)
array[++stackTop] = item; // Increments FIRST, then uses value

// Pop: post-decrement (stackTop--)
return array[stackTop--]; // Uses current value, THEN decrements
```

Why start at -1? When empty, there's no valid index | First push increments to 0 (first valid index) | `Size = stackTop + 1` works correctly



LINKED-LIST-BASED STACK

Dynamic size stack using a linked list. Top of stack = head of list.

```
template <class T>
class Stack {
private:
    struct Node {
        T data;
        Node* next;
        Node(const T& item, Node* ptr = NULL) {
            data = item;
            next = ptr;
        }
    };

    Node* topPtr; // Points to top of stack
    int count;    // Number of elements

public:
    Stack() {
        topPtr = NULL;
        count = 0;
    }

    ~Stack() {
        clear();
    }

    bool isEmpty() const {
        return (topPtr == NULL);
    }

    bool isFull() const {
        return false; // Linked list is never full (until out of memory)
    }

    int size() const {
        return count;
    }

    void push(const T& item) {
        Node* newNode = new Node(item);
        newNode->next = topPtr; // Link to current top
        topPtr = newNode;      // New node becomes top
        count++;
    }

    T pop() {
        if (isEmpty()) {
            throw "Stack Underflow";
        }

        T item = topPtr->data; // Save top data
        Node* temp = topPtr;
        topPtr = topPtr->next; // Remove top node
        delete temp;
        count--;
        return item;
    }

    T top() const {
        if (isEmpty()) {
            throw "Stack is empty";
        }
        return topPtr->data;
    }

    void clear() {
        while (!isEmpty()) {
            pop();
        }
    }
};
```

Why Push/Pop at Head?

Both operations are O(1) at the head of a singly linked list:

- Push: Insert at head (3 pointer changes)
- Pop: Delete from head (2 pointer changes)

If we pushed/popped from tail: Would need to traverse entire list (O(n) time)

Array vs Linked List Stack:

Feature	Array Stack	Linked List Stack
Size	Fixed (must specify max)	Dynamic (grows as needed)
Memory	Pre-allocated	Allocated per element
Speed	Slightly faster (no new)	Slightly slower (new overhead)
Overflow	Can be full	Never full (until RAM full)
Use When	Know max size	Size unpredictable

Common Stack Operations:
Check Balanced Parentheses:

```
bool isBalanced(string expr) {
    Stack<char> s;

    for (int i = 0; i < expr.length(); i++) {
        char ch = expr[i];

        if (ch == '(' || ch == '[' || ch == '{') {
            s.push(ch); // Push opening brackets
        }
        else if (ch == ')' || ch == ']' || ch == '}') {
            if (s.isEmpty()) {
                return false; // No matching opening
            }

            char top = s.pop();
            if ((ch == ')' && top != '(') ||
                (ch == ']' && top != '[') ||
                (ch == '}' && top != '{')) {
                return false;
            }
        }
    }

    return s.isEmpty(); // Stack should be empty at end
}
```

Reverse a String:

```
string reverse(string str) {
    Stack<char> s;

    for (int i = 0; i < str.length(); i++) {
        s.push(str[i]); // Push all characters
    }

    string reversed = "";
    while (!s.isEmpty()) {
        reversed += s.pop(); // Pop to build reversed string
    }

    return reversed;
}
```

Convert Decimal to Binary:

```
string decimalToBinary(int n) {
    if (n == 0) return "0";

    Stack<int> s;

    while (n > 0) {
        s.push(n % 2);
        n = n / 2; // Push remainders
    }

    string binary = "";
    while (!s.isEmpty()) {
        binary += to_string(s.pop()); // Pop to build binary string
    }

    return binary;
}
```





3. QUEUES

A queue is a First-In-First-Out (FIFO) data structure. Think of a line at a store - first person in line is first to be served.

Queue Operations:

- enqueue(item) add to rear
- dequeue() remove and return front
- front() view front
- rear()/back() view rear
- isEmpty()
- isFull() (array only)
- size()

When to Use Queues:

- Task scheduling
- Print spooler
- Breadth-first search
- Customer service
- Keyboard buffer

ARRAY-BASED QUEUE (Circular)

To avoid wasting space, we treat the array as circular - when we reach the end, we wrap around to the beginning.

```
template <class T>
class Queue {
private:
    T* array;
    int front, rear; // Index of front/rear element
    int maxSize;
    int count;      // Number of elements

public:
    Queue(int size = 100) {
        maxSize = size;
        array = new T[maxSize];
        front = 0;
        rear = -1;
        count = 0;
    }

    ~Queue() {
        delete[] array;
    }

    bool isEmpty() const {
        return (count == 0);
    }

    bool isFull() const {
        return (count == maxSize);
    }

    int size() const {
        return count;
    }

    void enqueue(const T& item) {
        if (isFull()) {
            throw "Queue Overflow";
        }
        rear = (rear + 1) % maxSize; // Move rear forward (with wraparound)
        array[rear] = item;
        count++;
    }

    T dequeue() {
        if (isEmpty()) {
            throw "Queue Underflow";
        }
        T item = array[front];
        front = (front + 1) % maxSize; // Move front forward (with wraparound)
        count--;
        return item;
    }

    T getFront() const {
        if (isEmpty()) {
            throw "Queue is empty";
        }
        return array[front];
    }

    T getRear() const {
        if (isEmpty()) {
            throw "Queue is empty";
        }
        return array[rear];
    }

    void clear() {
        front = 0;
        rear = -1;
        count = 0;
    }
};
```

Understanding Wraparound:

The modulo operation % maxSize creates circular behaviour:

```
// If maxSize = 5
rear = (rear + 1) % 5

// When rear = 3: (3 + 1) % 5 = 4
// When rear = 4: (4 + 1) % 5 = 0 ← wraps to start!
```

Alternative: No count variable:

```
class Queue {
private:
    T* array;
    int front, rear;
    int maxSize;

public:
    Queue(int size) {
        maxSize = size + 1; // Extra space needed!
        array = new T[maxSize];
        front = 0;
        rear = 0; // Both start at 0
    }

    bool isEmpty() const {
        return (front == rear);
    }

    bool isFull() const {
        return ((rear + 1) % maxSize == front);
    }

    void enqueue(const T& item) {
        if (isFull()) throw "Full";
        array[rear] = item;
        rear = (rear + 1) % maxSize;
    }

    T dequeue() {
        if (isEmpty()) throw "Empty";
        T item = array[front];
        front = (front + 1) % maxSize;
        return item;
    }
};
```

❏ **Why maxSize + 1?** Need one empty slot to distinguish full from empty | If front == rear, queue is empty | If (rear + 1) % maxSize == front, queue is full



LINKED-LIST-BASED QUEUE

Dynamic size queue using a linked list. Need pointers to BOTH front and rear.

```
template <class T>
class Queue {
private:
    struct Node {
        T data;
        Node* next;
        Node(const T& item) {
            data = item;
            next = NULL;
        }
    };

    Node* frontPtr; // Points to front of queue
    Node* rearPtr;  // Points to rear of queue
    int count;

public:
    Queue() {
        frontPtr = NULL;
        rearPtr = NULL;
        count = 0;
    }

    ~Queue() {
        clear();
    }

    bool isEmpty() const {
        return (frontPtr == NULL);
    }

    bool isFull() const {
        return false; // Never full
    }

    int size() const {
        return count;
    }

    void enqueue(const T& item) {
        Node* newNode = new Node(item);

        if (isEmpty()) {
            frontPtr = rearPtr = newNode; // First element
        } else {
            rearPtr->next = newNode;
            rearPtr = newNode; // Add to rear
        }
        count++;
    }

    T dequeue() {
        if (isEmpty()) {
            throw "Queue Underflow";
        }

        T item = frontPtr->data;
        Node* temp = frontPtr;
        frontPtr = frontPtr->next;

        if (frontPtr == NULL) {
            rearPtr = NULL; // If queue becomes empty, update rear
        }

        delete temp;
        count--;
        return item;
    }

    T getFront() const {
        if (isEmpty()) {
            throw "Queue is empty";
        }
        return frontPtr->data;
    }

    T getRear() const {
        if (isEmpty()) {
            throw "Queue is empty";
        }
        return rearPtr->data;
    }

    void clear() {
        while (!isEmpty()) {
            dequeue();
        }
    }
};
```

Why Two Pointers?

Without rear pointer (only front):

- Enqueue: O(n) - must traverse to end
- Dequeue: O(1) - remove from front

With both front and rear:

- Enqueue: O(1) - direct access to rear
- Dequeue: O(1) - direct access to front

Swift Note: Empty Queue:
When you dequeue the last element:

```
if (frontPtr == NULL) {
    rearPtr = NULL; // MUST set rear to NULL too!
}
```

Why? After dequeue, frontPtr becomes NULL but rearPtr still points to deleted node! Next enqueue would crash accessing rearPtr->next.

Array vs Linked List Queue:

Feature	Array Queue	Linked List Queue
Size	Fixed	Dynamic
Memory	Pre-allocated, circular	Per element
Complexity	Wraparound logic	Two pointers
Speed	Slightly faster	Slightly slower
Use When	Max size known	Size unpredictable

Static vs Dynamic Implementation:

```
// For static (array) stack/queue, constructor needs size:
Queue<int> q(50); // Max 50 elements

// For dynamic (linked list), no size needed:
Queue<int> q; // Unlimited size
```

Why does this matter for exams? If question says "implement with arrays", use array version | If question says "dynamically sized", use linked list version | If unclear, linked list is safer (never runs out of space)

www.swifttuition.com/tuks/cos110 for more free/premium shtuff 🍷



4. EXCEPTIONS

Exceptions are a way to handle errors without crashing your programme. When something goes wrong (divide by zero, file not found, array out of bounds), you can "throw" an exception and "catch" it to handle the error gracefully.

Why Use Exceptions?

Without exceptions:

- Error codes ambiguous (is -1 an error or valid result?)
- Must check return value every time
- Error handling mixed with normal code

With exceptions:

- Separate error handling from normal code
- Can skip multiple function levels
- More explicit about what went wrong

Basic Exception Syntax:

```
try {
    // Code that might throw an exception
    if (errorCondition) {
        throw exceptionObject;
    }
    // More code (skipped if exception thrown)
}
catch (ExceptionType parameter) {
    // Handle the exception
}
// Continue here after catch
```

Flow of execution:

1. Code in try block runs normally
2. If throw is executed: Immediately jump to matching catch block, Skip rest of try block
3. Execute code in catch block
4. Continue after catch block

What Can You Throw?

You can throw almost anything:

```
throw 404;           // Integer
throw "File not found"; // String literal (C-string)
throw string("Invalid input"); // String object
throw 'E';           // Character
throw true;           // Boolean
throw MyException("Error message"); // Object
```

Best practice: Throw objects (custom exception classes), not primitives.

Catching Exceptions:

```
try {
    // code that throws
}
catch (int errCode) {
    cout << "Error code: " << errCode;
}
catch (const char* msg) {
    cout << "Error message: " << msg;
}
catch (string msg) {
    cout << "String error: " << msg;
}
catch (...) { // Catch-all
    cout << "Unknown error";
}
```

📌 **Order matters!** Catches are checked in order, top to bottom | First matching catch is executed | Others are skipped | `catch(...)` catches everything - put it LAST



Custom Exception Classes:

```
class DivideByZeroException {
private:
    string message;
public:
    DivideByZeroException(string msg) {
        message = msg;
    }
    string getMessage() const {
        return message;
    }
};

// Usage:
int divide(int a, int b) {
    if (b == 0) {
        throw DivideByZeroException("Cannot divide by zero");
    }
    return a / b;
}

// Catching:
try {
    int result = divide(10, 0);
}
catch (DivideByZeroException& e) { // Catch by reference!
    cout << e.getMessage();
}
```

Why catch by reference (&)? Avoids copying the exception object | Can catch derived exception classes (polymorphism) | More efficient

Exception class with more info:

```
class ArrayIndexException {
private:
    int index;
    int size;
    string message;
public:
    ArrayIndexException(int idx, int sz) {
        index = idx;
        size = sz;
        message = "Index " + to_string(idx) +
            " out of bounds (size " + to_string(sz) + ")";
    }
    string getMessage() const { return message; }
    int getIndex() const { return index; }
    int getSize() const { return size; }
};

// Usage in array class:
T& operator[](int index) {
    if (index < 0 || index >= size) {
        throw ArrayIndexException(index, size);
    }
    return data[index];
}
```

Exception Hierarchies:

```
// Base exception class
class Exception {
protected:
    string message;
public:
    Exception(string msg) : message(msg) {}
    virtual string what() const { return message; }
};

// Derived exceptions
class InvalidIndexException : public Exception {
public:
    InvalidIndexException(string msg) : Exception(msg) {}
};

class ElementNotFoundException : public Exception {
public:
    ElementNotFoundException(string msg) : Exception(msg) {}
};

// Catching hierarchy:
try {
    // code that throws InvalidIndexException or OutOfBoundsException
}
catch (InvalidIndexException& e) {
    // Handle specific exception
}
catch (Exception& e) {
    // Catches any Exception or derived class
    // Put more general catches AFTER specific ones!
}
```

Why use inheritance? Can catch groups of related exceptions | Can catch base class to handle any derived exception | More organised exception handling



Exception Propagation:

If you don't catch an exception in the current function, it propagates up the call stack.

```
void functionC() {
    throw -1; // Throws exception
}

void functionB() {
    functionC(); // Exception passes through
}

void functionA() {
    try {
        functionB();
    }
    catch (int errCode) {
        cout << "Caught in A: " << errCode;
    }
}

int main() {
    functionA();
}

// Prints: "Caught in A: -1"
```

Re-throwing Exceptions:

```
void process() {
    try {
        // risky code
    }
    catch (Exception& e) {
        logError(e.what()); // Do some cleanup
        throw; // Re-throw the same exception
    }
}

void caller() {
    try {
        process();
    }
    catch (Exception& e) {
        cout << "Final handler: " << e.what();
    }
}
```

Note: throw; (no argument) re-throws the caught exception.

When NOT to Use Exceptions:

For normal control flow Don't use exceptions for logic that should be handled with if/else	For things you can check beforehand Check isEmpty() before pop() rather than catching exception	For performance-critical code Exceptions have overhead (stack unwinding)
--	---	--

Stack Unwinding:

When an exception is thrown, the programme "unwinds" the stack - local objects are destroyed automatically:

```
class Resource {
public:
    Resource() { cout << "Acquired\n"; }
    ~Resource() { cout << "Released\n"; }
};

void function() {
    Resource r; // Constructor called
    throw "Error"; // Destructor called automatically during unwinding!
}

int main() {
    try {
        function();
    }
    catch (const char* e) {
        cout << "Caught: " << e;
    }
}

// Output:
// Acquired
// Released ← Destructor called during unwinding
// Caught: Error
```

Common Exception Patterns:

```
// File operations
class FileNotFoundException : public Exception {
public:
    FileNotFoundException(string filename)
    : Exception("File not found: " + filename) {}
};

void openFile(string filename) {
    ifstream file(filename);
    if (!file.is_open()) {
        throw FileNotFoundException(filename);
    }
    // process file
}

// Array access
T& at(int index) {
    if (index < 0 || index >= size) {
        throw out_of_range("Index out of bounds");
    }
    return data[index];
}

// Stack/Queue operations
T pop() {
    if (isEmpty()) {
        throw runtime_error("Pop from empty stack");
    }
    // pop logic
}
```



5. STL (Standard Template Library)

The STL is a powerful library of template classes and functions. It provides common data structures (containers) and algorithms so you don't have to implement them yourself.

Why Use STL?

- Already implemented
- Well-tested
- Optimised
- Standard
- Time-saver

When NOT to use STL:

- Exam question says "implement your own"
- Need to understand how data structures work
- Learning purposes

STL Components:

- Containers (store data)
- Iterators (access elements)
- Algorithms (operate on containers)
- Function Objects (customise algorithms)

CONTAINERS

Sequence Containers - Store elements in linear sequence.

vector - Dynamic array

```
#include <vector>
using namespace std;

vector<int> v;           // Empty vector
vector<int> v(10);       // 10 elements, default value
vector<int> v(10, 5);    // 10 elements, all 5
vector<int> v = {1, 2, 3}; // Initialise with values

// Adding elements
v.push_back(42);         // Add to end: O(1) amortised
v.insert(v.begin(), 0);  // Insert at position: O(n)

// Accessing
v[0]                     // No bounds check: O(1)
v.at(0)                  // With bounds check: O(1)
v.front()               // First element: O(1)
v.back()                // Last element: O(1)

// Removing
v.pop_back();            // Remove last: O(1)
v.erase(v.begin());     // Remove at position: O(n)
v.clear();               // Remove all: O(n)

// Info
v.size()                 // Number of elements
v.empty()                // Check if empty
v.capacity()             // Current capacity

// Iterating
for (int i = 0; i < v.size(); i++) {
    cout << v[i];
}
```

When to use vector: Need random access by index | Mostly adding/removing at end | Don't know size in advance (unlike array)

Time complexity: Access: O(1) | Push/pop back: O(1) amortised | Insert/erase middle: O(n) | Search: O(n)

list - Doubly linked list

```
#include <list>

list<int> l;
list<int> l = {1, 2, 3};

// Adding
l.push_front(0);         // Add to front: O(1)
l.push_back(4);          // Add to back: O(1)
l.insert(it, val);       // Insert at iterator: O(1)

// Removing
l.pop_front();           // Remove first: O(1)
l.pop_back();            // Remove last: O(1)
l.erase(it);             // Remove at iterator: O(1)
l.remove(val);           // Remove all occurrences of value: O(n)

// Accessing
l.front()                // First element: O(1)
l.back()                 // Last element: O(1)
// NO random access - no l[i] or l.at(i)!

// Info
l.size()
l.empty()

// Iterating (must use iterator)
for (list<int>::iterator it = l.begin(); it != l.end(); ++it) {
    cout << *it;
}
```

When to use list: Frequent insert/delete in middle | Don't need random access | Need to maintain iterators after insert/delete

Time complexity: Access: O(n) - must traverse | Insert/erase with iterator: O(1) | Push/pop front/back: O(1) | Search: O(n)

deque - Double-ended queue

```
#include <deque>

deque<int> dq;
deque<int> dq = {1, 2, 3};

// Combines features of vector and list
dq.push_front(0);        // Add to front: O(1)
dq.push_back(4);         // Add to back: O(1)
dq.pop_front();          // Remove front: O(1)
dq.pop_back();           // Remove back: O(1)

// Random access (like vector)
dq[0]                    // O(1)
dq.at(1)                 // O(1) with bounds check
dq.front()
dq.back()

// Insert/erase in middle
dq.insert(dq.begin(), val); // O(n)
dq.erase(dq.begin());      // O(n)

dq.size()
dq.empty()
```

When to use deque: Need random access | Need fast insert/delete at BOTH ends | Alternative to vector if inserting at front

Time complexity: Access: O(1) | Push/pop front or back: O(1) | Insert/erase middle: O(n)



Container Adapters - Wrappers around other containers with restricted interfaces.

stack - LIFO

```
#include <stack>

stack<int> s; // Uses deque by default
stack<int, vector<int>>> s; // Use vector as underlying container

s.push(5);    // Add to top
s.pop();      // Remove top (void - doesn't return!)
s.top();      // View top
s.empty();    // Check if empty
s.size();     // Number of elements

// NO iterators - can't traverse a stack!
```

 **Important:** `pop()` returns void! Must call `top()` first to get value.

```
// WRONG:
int val = s.pop(); // Error! pop() is void

// CORRECT:
int val = s.top();
s.pop();
```

queue - FIFO

```
#include <queue>

queue<int> q; // Uses deque by default

q.push(5);    // Add to rear (enqueue)
q.pop();      // Remove from front (dequeue) - void!
q.front();    // View front
q.back();     // View rear
q.empty();
q.size();

// NO iterators
```

Same warning: `pop()` is void!

```
// WRONG:
int val = q.pop(); // Error!

// CORRECT:
int val = q.front();
q.pop();
```

priority_queue - Heap

```
#include <queue>

priority_queue<int> pq; // Max heap (largest on top)
priority_queue<int, vector<int>, greater<int>>> pq; // Min heap

pq.push(5); // Insert element
pq.pop();   // Remove top (void!)
pq.top();   // View top
pq.empty();
pq.size();

// Elements come out in sorted order
```




Associative Containers - Store sorted elements, fast lookup.
set - Sorted unique elements

```
#include <set>

set<int> s;
set<int> s = {3, 1, 4, 1, 5}; // Stores: 1, 3, 4, 5 (sorted, unique)

// Inserting
s.insert(2); // O(log n)
s.insert(2); // Duplicate ignored

// Searching
s.count(2) // Returns 1 if exists, 0 if not: O(log n)
s.find(2) // Returns iterator (or s.end()): O(log n)

// Removing
s.erase(2); // Remove element: O(log n)
s.erase(it); // Remove at iterator: O(1)

s.size()
s.empty()

// Iterating (always in sorted order!)
for (set<int>::iterator it = s.begin(); it != s.end(); ++it) {
    cout << *it << " "; // Prints: 1 3 4 5
}
```

When to use set: Need unique elements | Need elements sorted | Need fast search

Time complexity: O(log n) for insert, search, erase

multiset - Sorted elements (allows duplicates)

```
#include <set>

multiset<int> ms;
ms.insert(5);
ms.insert(5); // Allowed!
ms.count(5) // Returns 2

// Same operations as set, but allows duplicates
```

map - Sorted key-value pairs

```
#include <map>

map<int, string> m;
map<string, int> ages;

// Inserting
m[1] = "apple"; // Insert or update: O(log n)
m[2] = "banana";
m.insert(make_pair(3, "cherry")); // Alternative syntax

// Accessing
m[1] // Returns "apple": O(log n)
m.at(1) // With bounds check: O(log n)
m[99] // CREATES entry if doesn't exist!

// Searching
m.count(1) // Returns 1 if key exists, 0 if not
m.find(1) // Returns iterator: O(log n)
if (m.find(1) != m.end()) {
    cout << "Key 1 exists";
}

// Removing
m.erase(1); // Remove by key: O(log n)

m.size()
m.empty()

// Iterating (sorted by KEY)
for (map<int,string>::iterator it = m.begin(); it != m.end(); ++it) {
    cout << it->first << " = " << it->second << endl;
    // it->first is the key
    // it->second is the value
}
```

❏ **Important:** `m[key]` creates entry if doesn't exist! Use `m.find(key)` to just check.

When to use map: Associate keys with values | Need keys sorted | Need fast lookup by key

Time complexity: O(log n) for all operations

multimap - Allows duplicate keys

```
#include <map>

multimap<int, string> mm;
mm.insert(make_pair(1, "first"));
mm.insert(make_pair(1, "second")); // Same key!

// Cannot use [] operator with multimap
mm.count(1) // Returns 2
```



ITERATORS

Iterators are like pointers - they point to elements in containers and let you traverse/modify them.

Iterator Types:

- Input Iterator (Read forward only)
- Output Iterator (Write forward only)
- Forward Iterator (Read/write forward only)
- Bidirectional Iterator (Can go forward and backward)
- Random Access Iterator (Can jump to any position)

Container iterator types: vector, deque: Random access | list, set, map: Bidirectional | stack, queue: No iterators!

Basic Iterator Usage:

```
vector<int> v = {1, 2, 3, 4, 5};
vector<int>::iterator it; // Declare iterator

// Iterate through container
for (it = v.begin(); it != v.end(); ++it) {
    cout << *it << " "; // Dereference to get value
    *it = *it + 10;      // Can modify
}

// Range-based for (C++11)
for (int& val : v) {
    cout << val << " ";
}
```

Iterator Operations:

```
vector<int> v = {10, 20, 30, 40, 50};
vector<int>::iterator it = v.begin();

*it      // Returns 10 (Dereference)
++it;    // Move to next: 20
it++;    // Post-increment (prefer pre)
--it;    // Move to previous
it--;    // Post-decrement

// Random access (vector/deque only)
it += 3;  // Jump forward 3
it -= 2;  // Jump back 2
it + 5    // Iterator 5 ahead
it - 2    // Iterator 2 behind
it[3]     // Access relative element

// Comparison
it1 == it2
it1 != it2
it1 < it2 // Random access only
it1 > it2 // Random access only
it2 - it1 // Number of elements between (Distance)
```

Const Iterators - Can read but not modify elements:

```
vector<int> v = {1, 2, 3};
vector<int>::const_iterator cit;

for (cit = v.begin(); cit != v.end(); ++cit) {
    cout << *cit; // OK - reading
    *cit = 10;    // ERROR - can't modify
}

// C++11 way
for (cit = v.cbegin(); cit != v.cend(); ++cit) {
    /* ... */
}
```

Reverse Iterators - Traverse container backwards:

```
vector<int> v = {1, 2, 3, 4, 5};

for (vector<int>::reverse_iterator rit = v.rbegin();
     rit != v.rend(); ++rit) {
    cout << *rit << " "; // Prints: 5 4 3 2 1
}
```

Note: Still use ++ to move to next element (which is previous in forward order)!

Iterator Invalidation:

- **vector:** After push_back() if capacity changes | After insert() or erase() | After clear()
- **list:** Only invalidated when element is erased | Other iterators remain valid
- **map/set:** Only invalidated when element is erased | Inserting doesn't invalidate



ALGORITHMS

Need #include <algorithm>

Sorting:

```
#include <algorithm>
#include <vector>

vector<int> v = {5, 2, 8, 1, 9};

sort(v.begin(), v.end()); // Sort ascending: v is now {1, 2, 5, 8, 9}

sort(v.begin(), v.end(), greater<int>()); // Sort descending: v is now {9, 8, 5, 2, 1}

// Custom comparison
bool compare(int a, int b) {
    return a > b; // Descending
}
sort(v.begin(), v.end(), compare);

partial_sort(v.begin(), v.begin() + 3, v.end()); // First 3 elements sorted, rest unsorted

if (is_sorted(v.begin(), v.end())) {
    cout << "Sorted!";
} // Check if sorted
```

Time complexity: O(n log n)

Searching:

```
vector<int> v = {1, 2, 3, 4, 5, 6, 7, 8, 9};

// Linear search (unsorted OK)
vector<int>::iterator it = find(v.begin(), v.end(), 5);
if (it != v.end()) {
    cout << "Found at position " << (it - v.begin());
} else {
    cout << "Not found";
}

// Binary search (MUST be sorted first!)
sort(v.begin(), v.end());
bool found = binary_search(v.begin(), v.end(), 5); // Returns true/false

it = lower_bound(v.begin(), v.end(), 5); // Returns iterator to first >= 5
it = upper_bound(v.begin(), v.end(), 5); // Returns iterator to first > 5

int cnt = count(v.begin(), v.end(), 5); // Count occurrences

bool hasEven = any_of(v.begin(), v.end(), [](int x) { return x % 2 == 0; }); // Find if any element matches

bool allPositive = all_of(v.begin(), v.end(), [](int x) { return x > 0; }); // Find if all elements match
```

Modifying:

```
vector<int> v = {1, 2, 3, 4, 5};

reverse(v.begin(), v.end()); // v is now {5, 4, 3, 2, 1}

fill(v.begin(), v.end(), 0); // v is now {0, 0, 0, 0, 0}

vector<int> v2(5);
copy(v.begin(), v.end(), v2.begin()); // Copy

// Unique (remove consecutive duplicates)
v = {1, 1, 2, 2, 2, 3, 3, 1};
sort(v.begin(), v.end()); // Must sort first!
v.erase(unique(v.begin(), v.end()), v.end()); // v is now {1, 2, 3}

v.erase(remove(v.begin(), v.end(), 2), v.end()); // Remove specific value - Removes all 2s

replace(v.begin(), v.end(), 1, 10); // All 1s become 10s
```

Min/Max:

```
vector<int> v = {5, 2, 8, 1, 9};

vector<int>::iterator minIt = min_element(v.begin(), v.end());
cout << "Min: " << *minIt; // 1

vector<int>::iterator maxIt = max_element(v.begin(), v.end());
cout << "Max: " << *maxIt; // 9

int smaller = min(5, 10); // 5 (Min/max of two values)
int larger = max(5, 10); // 10
```

Permutations:

```
vector<int> v = {1, 2, 3};

do {
    for (int x : v) cout << x;
    cout << endl;
} while (next_permutation(v.begin(), v.end()));

// Prints all permutations: 123, 132, 213, 231, 312, 321
```

STL Quick Reference Table:

Container	Access	Insert	Erase	Search	Sorted
vector	O(1)	O(n)	O(n)	O(n)	No
list	O(n)	O(1)*	O(1)*	O(n)	No
deque	O(1)	O(n)	O(n)	O(n)	No
set	-	O(log n)	O(log n)	O(log n)	Yes
multiset	-	O(log n)	O(log n)	O(log n)	Yes
map	O(log n)	O(log n)	O(log n)	O(log n)	By key
multimap	O(log n)	O(log n)	O(log n)	O(log n)	By key

*With iterator



6. RECURSION

Recursion is when a function calls itself. It's a way to solve problems by breaking them into smaller versions of the same problem.

Essential Components:

- Base case (when to stop recursing)
- Recursive case (calls itself with simpler problem)
- Progress towards base case (each call gets closer to base case)

```
int factorial(int n) {
    if (n == 0) {
        return 1; // Base case
    }
    return n * factorial(n - 1); // Recursive case (progress: n gets smaller)
}
```

How Recursion Works - Example: factorial(3)

```
factorial(3)
= 3 * factorial(2)
= 2 * factorial(1)
= 1 * factorial(0)
= 1 ← base case
= 1 * 1 = 1
= 2 * 1 = 2
= 3 * 2 = 6
```

Call stack:

```
main()
→ factorial(3)
→ factorial(2)
→ factorial(1)
→ factorial(0) returns 1 ←
returns 1 ←
returns 2 ←
returns 6
```

Types of Recursion

Value-Returning Recursion - Function returns a value that combines with recursive result.

```
int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}

int power(int x, int n) {
    if (n == 0) return 1;
    return x * power(x, n - 1);
}

int sum(int arr[], int n) {
    if (n == 0) return 0;
    return arr[n-1] + sum(arr, n-1);
}

int fib(int n) {
    if (n <= 1) return n;
    return fib(n-1) + fib(n-2);
}
```

Void Recursion - Function performs action, doesn't return value.

```
// Print numbers 1 to n
void printNumbers(int n) {
    if (n == 0) return; // Base case
    printNumbers(n - 1); // Recursive call
    cout << n << " "; // Action after recursion
}
// printNumbers(3) prints: 1 2 3

// Print numbers n to 1
void printReverse(int n) {
    if (n == 0) return;
    cout << n << " "; // Action before recursion
    printReverse(n - 1);
}
// printReverse(3) prints: 3 2 1

// Write digits vertically
void writeVertical(int n) {
    if (n < 10) {
        cout << n << endl;
    } else {
        writeVertical(n / 10); // Process rest first
        cout << (n % 10) << endl; // Then print last digit
    }
}
// writeVertical(1234) prints:
// 1
// 2
// 3
// 4 (each on new line)
```

Key difference: Where you put the action (before or after recursive call) determines order!



Recursion on Arrays

```
// Find max in array
int findMax(int arr[], int n) {
    if (n == 1) return arr[0]; // Base: one element
    int maxRest = findMax(arr, n-1); // Max of rest
    return (arr[n-1] > maxRest) ? arr[n-1] : maxRest;
}

// Linear search
int search(int arr[], int n, int target) {
    if (n == 0) return -1; // Not found
    if (arr[n-1] == target) return n-1; // Found at last position
    return search(arr, n-1, target); // Search rest
}

// Binary search (recursive)
int binarySearch(int arr[], int left, int right, int target) {
    if (left > right) {
        return -1; // Not found
    }

    int mid = left + (right - left) / 2;

    if (arr[mid] == target) {
        return mid; // Found
    }
    if (target < arr[mid]) {
        return binarySearch(arr, left, mid-1, target); // Search left
    } else {
        return binarySearch(arr, mid+1, right, target); // Search right
    }
}

// Reverse array
void reverseArray(int arr[], int left, int right) {
    if (left >= right) return; // Base case

    int temp = arr[left]; // Swap ends
    arr[left] = arr[right];
    arr[right] = temp;

    reverseArray(arr, left+1, right-1); // Recurse on middle
}
```



Recursion on Linked Lists

```
// Count nodes
int countNodes(Node* head) {
    if (head == NULL) return 0; // Base case
    return 1 + countNodes(head->next);
}

// Print forward
void printForward(Node* head) {
    if (head == NULL) return;
    cout << head->data << " "; // Print first
    printForward(head->next); // Then recurse
}

// Print backward
void printBackward(Node* head) {
    if (head == NULL) return;
    printBackward(head->next); // Recurse first
    cout << head->data << " "; // Then print
}

// Find node with value
Node* find(Node* head, int value) {
    if (head == NULL) return NULL; // Not found
    if (head->data == value) return head; // Found
    return find(head->next, value); // Search rest
}

// Insert at end
Node* insertAtEnd(Node* head, int value) {
    if (head == NULL) {
        Node* newNode = new Node;
        newNode->data = value;
        newNode->next = NULL;
        return newNode; // Empty list - create new node
    }
    head->next = insertAtEnd(head->next, value); // Recursively insert at end of rest
    return head;
}

// Delete all nodes with value
Node* deleteValue(Node* head, int value) {
    if (head == NULL) return NULL; // Empty list

    head->next = deleteValue(head->next, value); // Recursively delete from rest

    if (head->data == value) { // If this node should be deleted
        Node* temp = head->next;
        delete head;
        return temp;
    }
    return head;
}

// Reverse linked list
Node* reverse(Node* head) {
    if (head == NULL || head->next == NULL) {
        return head; // Empty or single node
    }

    Node* newHead = reverse(head->next); // Reverse rest of list
    head->next->next = head; // Make next node point back to current
    head->next = NULL;
    return newHead;
}
```



Multiple Recursive Calls

Some problems require more than one recursive call.

```
// Fibonacci (two calls)
int fib(int n) {
    if (n <= 1) return n;
    return fib(n-1) + fib(n-2); // TWO recursive calls!
}

// Ackermann function (very deep recursion!)
int ackermann(int m, int n) {
    if (m == 0) {
        return n + 1;
    }
    if (n == 0) {
        return ackermann(m-1, 1);
    }
    return ackermann(m-1, ackermann(m, n-1)); // Nested recursion!
}

// Tower of Hanoi
void hanoi(int n, char from, char to, char aux) {
    if (n == 1) {
        cout << "Move disk from " << from << " to " << to << endl;
        return;
    }
    hanoi(n-1, from, aux, to); // Move n-1 to auxiliary
    cout << "Move disk from " << from << " to " << to << endl;
    hanoi(n-1, aux, to, from); // Move n-1 from aux to destination
}
```

Tail Recursion

When recursive call is the LAST thing function does (no operations after it).

```
// NOT tail recursive (multiplication after call)
int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n-1); // ← multiplication after
}

// Tail recursive version
int factorialTail(int n, int accum = 1) {
    if (n == 0) return accum;
    return factorialTail(n-1, n * accum); // ← nothing after call
}

// NOT tail recursive (addition after call)
int sum(int n) {
    if (n == 0) return 0;
    return n + sum(n-1); // ← addition after
}

// Tail recursive version
int sumTail(int n, int accum = 0) {
    if (n == 0) return accum;
    return sumTail(n-1, n + accum); // ← nothing after call
}
```

❏ **Why tail recursion matters:** Can be optimised by compiler to use constant stack space | Tail call optimisation converts to loop | More efficient (but C++ compilers don't always optimise)



Common Mistakes

1. No Base Case:

```
// WRONG - infinite recursion!
int bad(int n) {
    return n * bad(n-1); // Never stops! Stack overflow after ~1000-10000 calls
}

// Fix: Add base case
int factorial(int n) {
    if (n == 0) return 1; // Base case!
    return n * factorial(n-1);
}
```

2. Not Progressing Towards Base Case:

```
// WRONG - always calls with same value!
int bad(int n) {
    if (n == 0) return 1;
    return n * bad(n); // ← n doesn't change!
}

// Fix: Make sure parameter gets smaller
return n * factorial(n-1); // n-1 is closer to 0
```

3. Using Loops in Recursive Function:

If exam says "use recursion", don't use loops!

```
// WRONG - using loop defeats purpose of recursion
int sumArray(int arr[], int n) {
    int sum = 0;
    for (int i = 0; i < n; i++) { // NO LOOPS!
        sum += arr[i];
    }
    return sum;
}

// CORRECT - recursive
int sumArray(int arr[], int n) {
    if (n == 0) return 0;
    return arr[n-1] + sumArray(arr, n-1);
}
```

4. Using Global/Static Variables:

```
// WRONG - global variable is cheating
int count = 0;
int countNodes(Node* head) {
    if (head == NULL) return count;
    count++; // Modifying global!
    countNodes(head->next);
}

// CORRECT - pure recursion
int countNodes(Node* head) {
    if (head == NULL) return 0;
    return 1 + countNodes(head->next);
}
```

5. Wrong Return Type Integration:

```
// WRONG - doesn't combine results
int sum(int n) {
    if (n == 0) return 0;
    sum(n-1); // ← doesn't use return value!
    return n; // ← only returns n, not sum!
}

// CORRECT - combines results
int sum(int n) {
    if (n == 0) return 0;
    return n + sum(n-1); // ← combines n with recursive result
}
```



Tracing Recursion

To trace a recursive function: Write out each function call | Track parameters | Follow execution until base case | Track return values back up

Example: Trace factorial(3)

Call: factorial(3)
n=3, not base case
return 3 * factorial(2)
Call: factorial(2)
n=2, not base case
return 2 * factorial(1)
Call: factorial(1)
n=1, not base case
return 1 * factorial(0)
Call: factorial(0)
n=0, BASE CASE
return 1
return 1 * 1 = 1
return 2 * 1 = 2
return 3 * 2 = 6

Final answer: 6

Stack diagram:

factorial(0) → 1
↓
factorial(1) → 1
↓
factorial(2) → 2
↓
factorial(3) → 6
↓
main()

Recursion vs Iteration

Same problem, both ways:

Iterative factorial

```
int factorialIter(int n) {  
    int result = 1;  
    for (int i = 1; i <= n; i++) {  
        result *= i;  
    }  
    return result;  
}
```

Recursive factorial

```
int factorialRec(int n) {  
    if (n == 0) return 1;  
    return n * factorialRec(n-1);  
}
```

When to use recursion:

- Problem naturally recursive (tree traversal, divide-and-conquer)
- Code is simpler/clearer with recursion
- Exam question requires recursion
- Working with recursive data structures (linked lists, trees)

When to use iteration:

- Performance critical (recursion has overhead)
- Deep recursion might overflow stack
- Iterative solution is just as clear



COMMON PATTERNS ACROSS TOPICS

NULL/Empty Checks

Always check before accessing:

```
// Pointers
if (ptr != NULL) {
    /* Safe to use ptr->data */
}

// Containers
if (!container.empty()) {
    /* Safe to access front/back */
}

// Before any pointer dereference
if (head != NULL && head->next != NULL) {
    /* Safe to access head->next->data */
}
```

Edge Cases Checklist

For ANY data structure operation, test:

Empty structure

Empty list, empty stack, empty queue
Return appropriate value or throw exception

Single element

Might be both head and tail
Deletion makes structure empty

First element

Update head pointer
Special case for insert/delete

Last element

Update tail pointer (if tracked)
NULL checks for next pointer

Element not found

Return NULL or -1 or throw exception
Don't crash!

Full structure (bounded types)

Array-based stack/queue
Check before insert



Memory Management

Rule: Every new needs a matching delete.

```
// Allocate single object
Node* ptr = new Node;
// ... use ptr ...
delete ptr;
ptr = NULL; // Good practice

// Allocate array
int* arr = new int[10];
// ... use arr ...
delete[] arr; // Note the []!
arr = NULL;

// Destructor for linked list
~LinkedList() {
    while (head != NULL) {
        Node* temp = head;
        head = head->next;
        delete temp;
    }
}

// Copy constructor (deep copy)
LinkedList(const LinkedList& other) {
    head = NULL;
    Node* current = other.head;
    while (current != NULL) {
        insertAtTail(current->data); // Copy each node
        current = current->next;
    }
}
```

Common memory mistakes:

- Memory leak: Allocate but never delete
- Dangling pointer: Delete then access
- Double delete: Delete same memory twice
- Delete mismatch: new with delete[] or vice versa



Const Correctness

Use const for functions that don't modify:

```
class LinkedList {
public:
    int size() const { // Won't modify list
    int count = 0;
    Node* current = head;
    while (current != NULL) {
        count++;
        current = current->next;
    }
    return count;
}

    bool isEmpty() const {
    return (head == NULL); // Won't modify list
    }

    void print() const { // Won't modify list
    Node* current = head;
    while (current != NULL) {
        cout << current->data << " ";
        current = current->next;
    }
    }

    void insertAtHead(int value) {
    /* ... */
    } // WILL modify list - no const
};

// Function parameter
void printList(const LinkedList& list) { // Won't modify
    list.print(); // Only const methods allowed
}
```




Eo3 Swift TiPs

When Implementing Functions

Check edge cases FIRST:
Empty list, NULL pointers, single element, boundary conditions

Always check pointers before dereferencing:
if (ptr != NULL) before ptr->data

Use clear variable names:
Not just p, q - use current, nextNode, etc.

For recursion - find base case first:
What's the simplest case? How does bigger problem relate to smaller?

When Tracing Code

Draw pictures for linked lists:
Visual representation of pointers helps immensely

Track all pointer changes:
Write down state after each operation

Draw call stack for recursion:
Show each function call and return value

Test with edge cases:
What if list is empty? What if n = 0? What if element not found?

When Debugging Broken Code

Common bugs to look for:

Missing NULL checks:

`while (current->next != NULL)` crashes if current is NULL!
Fix: `while (current != NULL && current->next != NULL)`

Wrong order of operations:

`head = newNode; newNode->next = head;` loses list!
Fix: Link first, then update head

Off-by-one errors:

`for (int i = 0; i < n-1; i++)` vs `for (int i = 0; i < n; i++)`

Missing base case in recursion:

Infinite recursion! Add `if (n == 0) return 1;`

Memory leaks:

`head = head->next;` loses old head! Save in temp, then delete

For doubly linked lists - forgetting to update both pointers:

Must update both next and prev

For circular lists - checking for NULL:

Never NULL in circular list! Use `while (current != start)`

For exceptions - wrong catch order:

Specific catches first, `catch(...)` last



Time Savers

Use STL when allowed:

- `vector<int> v;` instead of manual array management
- `stack<int> s;` instead of implementing from scratch

Know iterator syntax:

```
for (vector<int>::iterator it = v.begin(); it != v.end(); ++it)
```

Remember STL quirks:

- `pop()` doesn't return value! Use `top()/front()` first
- `m[key]` creates entry! Use `find()` to check

Common patterns:

- Swap: `int temp = a; a = b; b = temp;`
- Find second-to-last: `while (current->next->next != NULL)`
- Wraparound: `index = (index + 1) % size`

Formula Reference

Linked list size (without counter):

Traverse and count

Array stack:

Empty: `stackTop == -1`

Full: `stackTop == maxSize - 1`

Size: `stackTop + 1`

Circular queue:

Wraparound: `(index + 1) % maxSize`

Empty: `count == 0`

Full: `count == maxSize`

Factorial:

Base: `n == 0` returns 1

Recursive: `n * factorial(n-1)`

Fibonacci:

Base: `n <= 1` returns `n`

Recursive: `fib(n-1) + fib(n-2)`

Binary search:

Mid: `left + (right - left) / 2`

Search left: `right = mid - 1`

Search right: `left = mid + 1`



FINAL CHECKLIST

Before the exam, make sure you can:

Linked Lists:

- Draw a linked list with pointers
- Insert at head, tail, middle
- Delete from head, tail, middle
- Traverse and find elements
- Handle circular and doubly linked lists
- Check for NULL before dereferencing

Stacks:

- Implement with array (with pre/post increment)
- Implement with linked list
- Know when stack is empty/full
- Remember stackTop = -1 means empty

Queues:

- Implement circular queue with array
- Implement with linked list (front and rear pointers)
- Handle wraparound with modulo
- Update both pointers in linked list version

Exceptions:

- Write try-catch-throw blocks
- Create custom exception classes
- Catch in correct order (specific to general)
- Understand exception propagation

STL:

- Use vector, list, deque, set, map
- Write iterator loops
- Use algorithms (sort, find, count)
- Know which containers allow what operations

Recursion:

- Identify base case and recursive case
- Trace recursion with call stack
- Write recursive functions for arrays and linked lists
- Avoid loops, globals, and static variables in recursive functions

General:

- Always check for NULL/empty
- Test edge cases
- Match new with delete
- Use const for read-only functions



Good luck on your
EO3! 🥰